GDB Debugging Command Sheet

CPU, OpenMP and GPU debugging | Advanced Programming - HPC | Monojyotsna Koch

1. Compile for debugging

`gcc -g -O0 -o prog prog.c`	-g adds symbols, -O0 turns off optimisation
`g++ -g -O0 -o prog prog.cpp`	Same for C++
`gcc -g -O0 -fopenmp ...`	Enable OpenMP
`-fsanitize=thread`	ThreadSanitizer: finds data races
`-fsanitize=address`	AddressSanitizer: memory errors
`nvcc -g -G -o app app.cu`	CUDA: -G debug device code (no timing!)
`hipcc -g -O0 -o app app.hip`	HIP on AMD GPUs

2. Start and exit

`gdb ./prog`	Start GDB on a program
`gdb --args ./prog a b`	Start with program arguments
`run (r) [args]`	Run the program
`start`	Run and stop at main
`quit (q)`	Exit GDB
`help CMD`	Help on a command

3. Breakpoints

`break main`	Stop at a function
`break file.c:6`	Stop at a line (or: break 6)
`tbreak main`	One-time breakpoint
`break f if n == 13`	Stop only when condition is true
`condition N expr`	Add a condition to breakpoint N
`info breakpoints`	List breakpoints and hit counts
`disable N / enable N`	Turn breakpoint N off / on
`delete N`	Remove breakpoint N (no N = all)

4. Run and step

`next (n)`	Run one line, step over calls
`step (s)`	Run one line, step into calls
`continue (c)`	Resume to the next breakpoint
`finish`	Run until this function returns
`until / advance 25`	Run to a line
`kill`	End the program being debugged
`list (l)`	Show source around the current line
`layout src`	Text UI with source (Ctrl-X A exits)

5. Inspect data

`print x (p x)`	Show a variable
`print *ptr`	Value the pointer points to
`print arr[0]@5`	Show 5 array elements
`print/x x`	Show in hex
`info locals`	All local variables
`info args`	Arguments of this function
`display x`	Print x at every stop
`set var x = 5`	Change a variable
`whatis x / ptype x`	Show a type
`x/8dw arr`	Examine memory: 8 ints, decimal

6. Call stack

`backtrace (bt)`	Show the call chain
`bt full`	Chain with local variables
`frame N`	Select frame N
`up / down`	Move to the caller / callee
`info frame`	Details of the selected frame

7. Watchpoints

`watch x`	Stop when x changes
`rwatch x / awatch x`	Stop when x is read / read or written
`info watchpoints`	List watchpoints

8. Crashes and running processes

`ulimit -c unlimited`	Allow a core file (shell command)
`gdb ./prog core`	Open a core dump, then run bt
`gdb -p PID`	Attach to a running or hung process
`~/.gdbinit`	Commands run at every GDB start

Prompt is (gdb); commands shown without it. Shortcuts in brackets. Date: 09.10.26

GDB Debugging Command Sheet

CPU, OpenMP and GPU debugging | Advanced Programming - HPC | Monojyotsna Koch

9. Threads and OpenMP

`OMP_NUM_THREADS=4 gdb ./prog`	Set thread count for the run
`info threads`	List threads and where they are
`thread N`	Switch to thread N
`thread apply all bt`	Backtrace of every thread
`thread apply all info locals`	Locals of every thread
`set scheduler-locking on`	Only the current thread runs
`set scheduler-locking off`	All threads run again (default)

10. CUDA-GDB (NVIDIA, CUDA)

`cuda-gdb ./app`	Start the CUDA debugger
`break my_kernel`	Stop at a kernel
`info cuda kernels`	List running kernels
`info cuda threads`	List GPU threads
`cuda block 1,0,0 thread 3,0,0`	Focus on one block and thread
`set cuda memcheck on`	Check device memory accesses
`compute-sanitizer ./app`	Find memory and race errors

11. ROCgdb (AMD, HIP)

`rocgdb ./app`	Start the ROCm debugger
`break my_kernel`	Stop at a kernel
`info threads`	See host and GPU threads
`thread N / bt / print`	Same GDB workflow as on the CPU

12. Common signals

`SIGSEGV`	Bad memory access, e.g. NULL pointer
`SIGFPE`	Arithmetic error, e.g. divide by zero
`SIGABRT`	Program aborted, e.g. failed assert

13. Run and check without GDB

`OMP_NUM_THREADS=4 ./prog`	Set threads for one run
`export OMP_NUM_THREADS=4`	Set threads for this shell
`nproc`	Number of cores available
`time ./prog`	Measure run time
`valgrind ./prog`	Memory checker (much slower)

Quick tips

- Press Enter to repeat the last command.
- Tab completes commands and names.
- **No symbol "i" in current context**: you are in the wrong frame or scope. Use **info locals** or **frame N**.
- A breakpoint inside a hot loop stops every pass: use **delete**, **disable** or a condition.
- Races depend on timing: a program that passes once is not proven correct.